PSLQ ALGORITHM FOR CERTIFYING RELATIONS

ABHIRAM KIDAMBI, FOR TORIC MZV COLLABORATION

Contents

1. Pari Code 1
2. Pseucocode 7

For any issues, contact kidambi@mis.mpg.de.

1. Pari Code

```
\\ Implementation of the short PSLQ algorithm over Q
\\ Based on arXiv:1809.06063v1 [math.NT]
\\ Extending the PSLQ algorithm to algebraic integer relations by Matthew Skerritt and Paul Vrbik
\\ Code written by: Abhiram Kidambi
\\ Updated 13 May 2026

issqmat(M)={
return(type(M)=="t_MAT"&&matsize(M)[1]==matsize(M)[2]);
};

istrapezoidal(M)={
my(sz=matsize(M));
return(sz[1]>sz[2]);
};

diagonalLTM(M)={
my(sz=matsize(M),r);
r=min(sz[1],sz[2]);
return(vector(r,i,M[i,i]));
};

rowswap(M,m)={
my(tmp);
tmp=M[m,];
M[m,]=M[m+1,];
M[m+1,]=tmp;
return(M);
};

colswap(M,m)={
my(tmp);
tmp=M[,m];
M[,m]=M[,m+1];
M[,m+1]=tmp;
return(M);
};
```

```
37
38 nint(x)={
39 return(round(x));
40 };
41
42 selm(gamma,H)={
43 my(sz,ncols,vals,m);
44 sz=matsize(H);
45 ncols=sz[2];
46 vals=vector(ncols,i,gamma^i*abs(H[i,i]));
47 vecmax(vals,&m);
48 return(m);
49 };
50
51 buildH(n,s,y)={
52 my(H);
53 H=matrix(n,n-1);
54 for(i=1,n,
55 for(j=1,n-1,
56 if(i<j,
57 H[i,j]=0,
58 if(i==j,
59 H[i,j]=s[i+1]/s[i],
60 H[i,j]=-y[i]*y[j]/(s[j]*s[j+1])
61 )
62 );
63 );
64 );
65 return(H);
66 };
67
68 hermitered(H,A,B,{eps=0})={
69 my(n,q);
70 n=matsize(A)[1];
71 if(eps==0,
72 eps=10^(-default(realprecision)+20);
73 );
74 for(i=2,n,
75 forstep(j=i-1,1,-1,
76 if(abs(H[j,j])<eps,
77 next;
78 );
79 q=round(H[i,j]/H[j,j]);
80 if(q==0,
81 next;
82 );
83 for(k=1,j,
84 H[i,k]-=q*H[j,k];
85 );
86 A[i,]-=q*A[j,];
87 B[,j]+=q*B[,i];
88 );
89 );
90 return([H,A,B]);
91 };
92
```

```
fixH(H,m)={
my(a,b,r,c,s,h0,h1);
a=H[m,m];
b=H[m,m+1];
r=sqrt(a^2+b^2);
if(r==0,
return(H);
);
c=a/r;
s=b/r;
for(i=m,matsize(H)[1],
h0=H[i,m];
h1=H[i,m+1];
H[i,m]=c*h0+s*h1;
H[i,m+1]=-s*h0+c*h1;
);
return(H);
};

pslqtau(gamma)={
my(gcrit,tau);
gcrit=sqrt(4/3);
if(gamma<=gcrit,
error("pslqtau: gamma must be strictly larger than sqrt(4/3).");
);
tau=1/sqrt(1/gamma^2+1/4);
return(tau);
};

pslqiterbound(vec,Mrel,{gamma=1.2})={
my(n,tau,N);
n=#vec;
if(n<2,
error("pslqiterbound: dimension of vec must be at least 2.");
);
if(Mrel<=0,
error("pslqiterbound: relation norm bound Mrel must be positive.");
);
tau=pslqtau(gamma);
N=binomial(n,2)*log(gamma^(n-1)*Mrel)/log(tau);
return(max(1,ceil(N)));
};

pslq(vec,{itermax=1000},{flag=0},{gamma=sqrt(4/3)+0.01},{tol=0},{verbose=1})
    ={
my(n,svec,yvec,H,A,B,m,iter);
my(res,normv,eps,residual,relation);
n=#vec;
iter=0;
if(n<2,
error("pslq: vector must have length at least 2.");
);
if(gamma<=sqrt(4/3),
error("pslq: gamma must be strictly larger than sqrt(4/3).");
);
if(tol==0,
```

```
tol=10^(-floor(0.75*default(realprecision)));
);
eps=10^(-default(realprecision)+20);
normv=sqrt(vec*vec~);
if(normv==0,
error("pslq: zero input vector.");
);
vec=vec/normv;
svec=vector(n,k,sqrt(sum(j=k,n,vec[j]^2)));
yvec=vec/svec[1];
svec=svec/svec[1];
H=buildH(n,svec,yvec);
A=matid(n);
B=matid(n);
while(iter<itermax,
if(verbose,
printf("%sPSLQ iteration %d/%d",Strchr(13),iter+1,itermax);
);
res=hermitered(H,A,B,eps);
H=res[1];
A=res[2];
B=res[3];
m=selm(gamma,H);
H=rowswap(H,m);
A=rowswap(A,m);
B=colswap(B,m);
if(m<n-1,
H=fixH(H,m);
);
for(j=1,n,
relation=B[,j];
residual=abs(vec*relation);
if(residual<tol,
if(verbose,
printf("%sRelation found at PSLQ iteration %d/%d, column %d\n",Strchr(13),
    iter+1,itermax,j);
print("Evaluation of relation = ",residual);
print("Relation = ");
print(relation);
);
if(flag==1,
return(1),
return(relation)
);
);
);
iter++;
);
if(verbose,
print("");
);
if(flag==1,
return(0),
error("pslq: no relation found within itermax.");
);
};
```

```
203
204 pslqgamma(vec,Mrel,{flag=0},{gamma=1.2},{tol=0},{verbose=1})={
205 my(itermax);
206 if(gamma<=sqrt(4/3),
207 error("pslqgamma: gamma must be strictly larger than sqrt(4/3).");
208 );
209 itermax=pslqiterbound(vec,Mrel,gamma);
210 return(pslq(vec,itermax,flag,gamma,tol,verbose));
211 };
212
213 pslqbounded(vec,Mrel,{flag=0},{gamma=1.2},{tol=0},{verbose=1})={
214 my(N);
215 N=pslqiterbound(vec,Mrel,gamma);
216 print("Dimension n       = ",#vec);
217 print("Relation norm M   = ",Mrel);
218 print("gamma             = ",gamma);
219 print("tau               = ",pslqtau(gamma));
220 print("Iteration bound N = ",N);
221 return(pslq(vec,N,flag,gamma,tol,verbose));
222 };
```

```
1
2 \\ Example Run
3
4 mzvec(n)={
5 my(v);
6 v=vector(n-1);
7 v[1]=zetamult([n]);
8 for(k=1,n-2,
9 v[k+1]=zetamult([n-k,k]);
10 );
11 return(v);
12 };
13
14 mzvec_23(n)={
15 my(v);
16 v=List();
17 for(len=1,n\2,
18 forvec(x=vector(len,i,[2,3]),
19 if(sum(i=1,len,x[i])==n,
20 listput(v,zetamult(x));
21 );
22 );
23 );
24 return(Vec(v));
25 };
26
27 pairwise_products(n)={
28 my(v,res);
29 v=mzvec_23(n);
30 res=List();
31 for(i=1,#v,
32 for(j=i,#v,
33 listput(res,v[i]*v[j]);
34 );
35 );
```

```
return(Vec(res));
};



\p 1200
v = pairwise_products(10);
Mrel = #v * 10^23;
gm=sqrt(4/3) + 1;
rel = pslqbounded(v, Mrel, 0, gm);

/* The output we get is

Dimension n        = 28
Relation norm M    = 28000000000000000000000000
gamma              =
   2.154700538379251529018297561003914911295203502540253752037204652967955344605866691

tau                =
   1.465856078100299879916067186752418997760120138691784752114785078478944868996533798

Iteration bound N = 76125
Relation found at PSLQ iteration 14897/76125, column 2
Residual = 1.6290979968137236385 E-1194
Relation =
[4113414477444870000, 11168840049596326200, 9351168337459974000,
   7780916664213930000, 10361147299189752800, 6658321356517230000,
   -108594930391617472000, 7679117027914369203, 12773030553349878120,
   10515846056424462900, 14171810536210292214, 8943099438566381400,
   -67275182473279359360, 5329635054385411200, 8822017078080588000,
   11812627355170667280, 7527773153838888000, -86109773838803347200,
   3679026192079087500, 9748921500042269100, 6312640813867440000,
   -101607970288175904000, 6528625962706643283, 8317541161037889600,
   -53086451554806691840, 2715496736380560000, -114012872158098144000,
   428071864754152140800]~


This is the same as lindep(v)
*/
```

2. Pseucocode

Algorithm 1 Helper Functions for Short PSLQ over $\mathbb{Q}$

Require: Real vector $\mathbf{x} = (x_1, \ldots, x_n)$, iteration bound itermax
Ensure: Integer relation vector $\mathbf{r} \in \mathbb{Z}^n$ such that $\mathbf{x} \cdot \mathbf{r} = 0$

$\gamma \leftarrow \sqrt{4/3}, \quad \varepsilon \leftarrow 10^{-80}$

function BuildH($n, \mathbf{s}, \mathbf{y}$)
    Initialize $H \in \mathbb{R}^{n \times (n-1)}$
    for $i = 1$ **to** n **do**
        for $j = 1$ **to** $\min(i, n-1)$ **do**
            if $i = j$ **then**
                $H_{ij} \leftarrow s_{i+1}/s_i$
            else if $i > j$ **then**
                $H_{ij} \leftarrow -(y_i y_j)/(s_j s_{j+1})$
            else
                $H_{ij} \leftarrow 0$
            end if
        end for
    end for
    return H
end function

function HermiteReduce(H, A, B)
    for $i = 2$ **to** n **do**
        for $j = i - 1$ **down to** 1 **do**
            $q \leftarrow \lfloor H_{ij}/H_{jj} \rceil$
            if $q \neq 0$ **then**
                for $k = 1$ **to** j **do**
                    $H_{ik} \leftarrow H_{ik} - qH_{jk}$
                end for
                $\mathrm{Row}_i(A) \leftarrow \mathrm{Row}_i(A) - q \cdot \mathrm{Row}_j(A)$
                $\mathrm{Col}_j(B) \leftarrow \mathrm{Col}_j(B) + q \cdot \mathrm{Col}_i(B)$
            end if
        end for
    end for
    return (H, A, B)
end function

function FixH(H, m)
    $a \leftarrow H_{mm}, \quad b \leftarrow H_{m,m+1}, \quad r \leftarrow \sqrt{a^2 + b^2}$
    $c \leftarrow a/r, \quad s \leftarrow b/r$
    for $i = m$ **to** n **do**
        $\alpha \leftarrow H_{im}, \quad \beta \leftarrow H_{i,m+1}$
        $H_{im} \leftarrow c\alpha + s\beta$
        $H_{i,m+1} \leftarrow -s\alpha + c\beta$
    end for
    return H
end function

Algorithm 2 Short PSLQ over $\mathbb{Q}$

Main Execution:
$\mathbf{x} \leftarrow \mathbf{x}/\|\mathbf{x}\|_2$
for $k = 1$ **to** n **do**
    $s_k \leftarrow \sqrt{\sum_{j=k}^{n} x_j^2}$
end for
$\mathbf{y} \leftarrow \mathbf{x}/s_1, \quad \mathbf{s} \leftarrow \mathbf{s}/s_1$
$H \leftarrow \textsc{BuildH}(n, \mathbf{s}, \mathbf{y}), \quad A \leftarrow I_n, \quad B \leftarrow I_n, \quad \text{iter} \leftarrow 0$
while iter $<$ itermax **do**
    $(H, A, B) \leftarrow \textsc{HermiteReduce}(H, A, B)$
    $m \leftarrow \text{argmax}_{1 \le i < n}\{\gamma^i |H_{ii}|\}$
    Swap rows $m, m+1$ in H and A; Swap columns $m, m+1$ in B
    if $m < n - 1$ **then**
        $H \leftarrow \textsc{FixH}(H, m)$
    end if
    for $j = 1$ **to** n **do**
        if $|\mathbf{x} \cdot B_{:,j}| < \varepsilon$ **then**
            return $B_{:,j}$
        end if
    end for
    iter $\leftarrow$ iter $+ 1$
end while
Error: No relation found